PROJECT REPORT

# Implement Client Script and UI Policy

Enforcing Data Integrity on ServiceNow Incident Records

## Table of Contents


| | |
|---|---|
| 1 | Introduction |
| 2 | Ideation Phase |
| 3 | Requirement Analysis |
| 4 | Project Design |
| 5 | Project Planning & Scheduling |
| 6 | Functional and Performance Testing |
| 7 | Results |
| 8 | Advantages & Disadvantages |
| 9 | Conclusion |
| 10 | Future Scope |
| 11 | Appendix |

# 1. Introduction

## 1.1 Project Overview

ServiceNow is a cloud-based platform for IT Service Management (ITSM). The **Incident** table is one of its most heavily used tables, where users report and track disruptions to services. Poor or inconsistent data on incident forms leads to delayed resolution, inaccurate reporting and weak SLA tracking.

This project implements **UI Policies** and **Client Scripts** on the Incident form to enforce data integrity. Fields become mandatory, read-only or hidden depending on conditions; values are auto-populated; and form submission is blocked when required conditions are not met.

## 1.2 Purpose

- Ensure complete and accurate data entry on Incident records.
- Reduce manual errors by automating field behaviour and auto-population.
- Provide immediate, client-side feedback to users without a server round-trip.
- Demonstrate practical use of Incident Management, UI Policy, UI Policy Actions, Client Scripts and Form Validation.

# 2. Ideation Phase

## 2.1 Problem Statement

Incident forms are frequently submitted with missing or inconsistent information, for example a closed incident without close notes, or a high-priority incident without a description. Because there is no real-time enforcement, support teams spend extra time chasing details, and reports become unreliable. A mechanism is needed that guides users at the moment of data entry and prevents invalid submissions.

## 2.2 Empathy Map Canvas

| Says | Thinks |
|---|---|
| "The incident has no details, I can't work on it."<br>"Why was this closed without notes?" | Clean data would save me time.<br>The form should tell me what is required. |
| **Does** | **Feels** |
| Calls the caller back for missing details; manually fixes records; re-checks fields before closing. | Frustrated by rework; worried about SLA breaches; relieved when forms guide them. |

## 2.3 Brainstorming

Ideas considered by the team for solving the data-quality problem:

| Idea | Approach | Decision |
|---|---|---|
| Make fields mandatory by condition | UI Policy with Mandatory action | Selected |
| Show/hide or lock fields by state | UI Policy Actions (Visible / Read-only) | Selected |

| Idea | Approach | Decision |
|---|---|---|
| Auto-fill fields (e.g. based on category) | onChange Client Script with g_form.setValue | Selected |
| Block submit if conditions fail | onSubmit Client Script returning false | Selected |
| Server-side Business Rules only | Validation after submission | Kept for future scope |

# 3. Requirement Analysis

## 3.1 Customer Journey Map

| Stage | User Action | Experience / Pain Point | Opportunity |
|---|---|---|---|
| Awareness | Service disruption noticed | Unsure what details to provide | Clear form guidance |
| Logging | Opens new Incident form | Many optional fields cause confusion | Highlight mandatory fields via UI Policy |
| Entry | Fills short description, category, priority | Mistakes and omissions | Auto-populate and validate with Client Scripts |
| Submission | Clicks Submit | Incomplete record saved | Block submission with clear message |
| Resolution | Agent works the incident | Needs complete data | Consistent, reliable records |
| Closure | Resolves/closes incident | Closed without notes | Mandatory close notes and code |

## 3.2 Solution Requirement

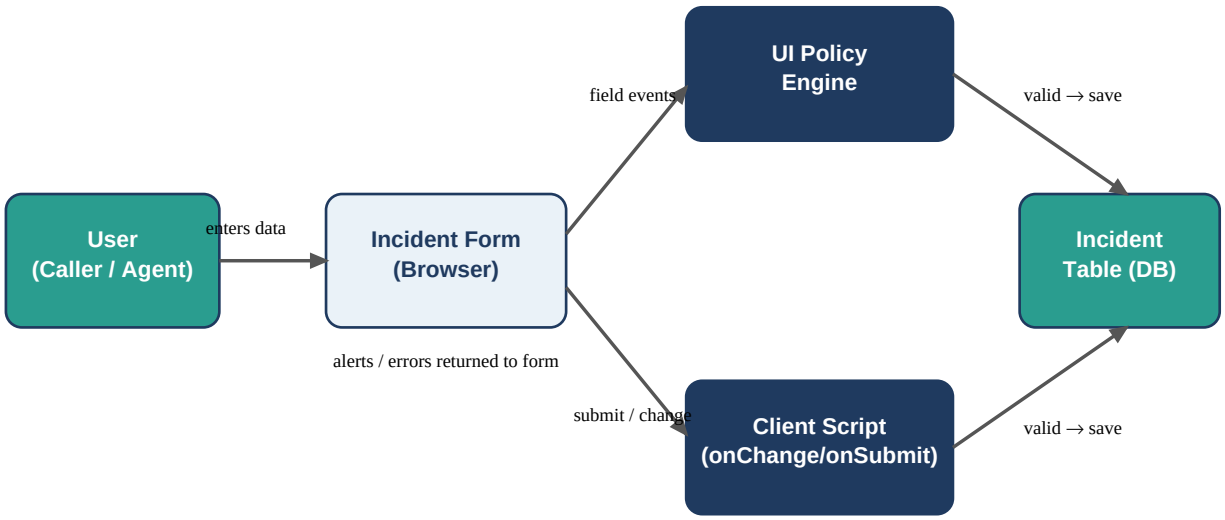
**Functional requirements**

- Make fields mandatory based on conditions (e.g. State = Resolved requires Close notes).
- Hide, show or set read-only fields based on field values.
- Auto-populate fields when a related field changes.
- Display alert/error messages for invalid input.
- Prevent form submission when validation fails.

**Non-functional requirements**

- **Performance:** client-side logic executes instantly without noticeable delay.
- **Usability:** messages are clear and fields are visually marked.
- **Maintainability:** scripts are commented and named consistently.
- **Compatibility:** works across supported browsers and the Service Operations Workspace/classic UI where applicable.
- **Security:** only authorised developers (admin role) can modify policies and scripts.

## 3.3 Data Flow Diagram




The user enters data in the Incident form. UI Policies and Client Scripts evaluate field events in the browser. Valid data is saved to the Incident table; invalid data triggers messages back to the form.

### 3.4 Technology Stack

| Layer | Technology |
|---|---|
| Platform | ServiceNow (Personal Developer Instance) |
| Module | Incident Management (incident table) |
| Configuration | UI Policy and UI Policy Actions |
| Scripting | Client Scripts (JavaScript) using g_form and g_user APIs |
| Tooling | Studio / System Definition > Client Scripts, Form Designer, Script Debugger |

## 4. Project Design

### 4.1 Problem Solution Fit

| Problem | Solution | Fit |
|---|---|---|
| Missing mandatory details | Conditional mandatory fields (UI Policy) | Guarantees required data at entry |
| Inconsistent field values | Auto-populate (Client Script) | Removes manual errors |
| Irrelevant fields clutter the form | Hide / read-only (UI Policy Actions) | Simpler, focused form |
| Invalid records get saved | onSubmit validation | Blocks bad data before it reaches the database |

### 4.2 Proposed Solution

Configure a set of UI Policies for declarative behaviour and complement them with Client Scripts for logic that needs scripting:

| Component | Table / Type | Behaviour |
|---|---|---|
| UI Policy 1 | Incident - State = Resolved/Closed | Close notes and Close code become mandatory |
| UI Policy 2 | Incident - Priority = 1 (Critical) | Description becomes mandatory; business impact field visible |
| UI Policy 3 | Incident - State = Closed | Key fields set to read-only |
| Client Script 1 | onChange - Category | Auto-populates related field values / shows info message |
| Client Script 2 | onSubmit | Validates short description length; returns false to stop submission |
| Client Script 3 | onLoad | Sets defaults and focuses key field on new record |

### 4.3 Solution Architecture




Presentation layer (client-side) -> Data layer (server)

The solution runs entirely in the presentation tier. UI Policies and Client Scripts intercept user events on the Incident form; only data that passes validation is committed to the Incident table.

## 5. Project Planning & Scheduling

### 5.1 Project Planning

| Phase | Activities | Duration |
|---|---|---|
| Sprint 1 | Requirement study, instance setup, problem statement | Week 1 |
| Sprint 2 | Design of UI Policies and UI Policy Actions | Week 2 |
| Sprint 3 | Develop Client Scripts (onLoad, onChange, onSubmit) | Week 3 |
| Sprint 4 | Testing, bug fixing, documentation and report | Week 4 |

## 6. Functional and Performance Testing

### 6.1 Performance Testing

| # | Test Case | Expected Result | Status |
|---|---|---|---|
| 1 | Set State = Resolved without close notes | Close notes flagged mandatory; save blocked | Pass |

| # | Test Case | Expected Result | Status |
|---|---|---|---|
| 2 | Set Priority = 1 with empty description | Description becomes mandatory | Pass |
| 3 | Change Category | Related field auto-populated | Pass |
| 4 | Submit with very short description | Error message shown; submission stopped | Pass |
| 5 | Open Closed incident | Locked fields are read-only | Pass |
| 6 | Form load time with all policies active | No noticeable delay (client-side execution) | Pass |

# 7. Results

## 7.1 Output Screenshots

The screenshots below were captured from the ServiceNow instance and show the UI Policies and Client Scripts working on the Incident form.

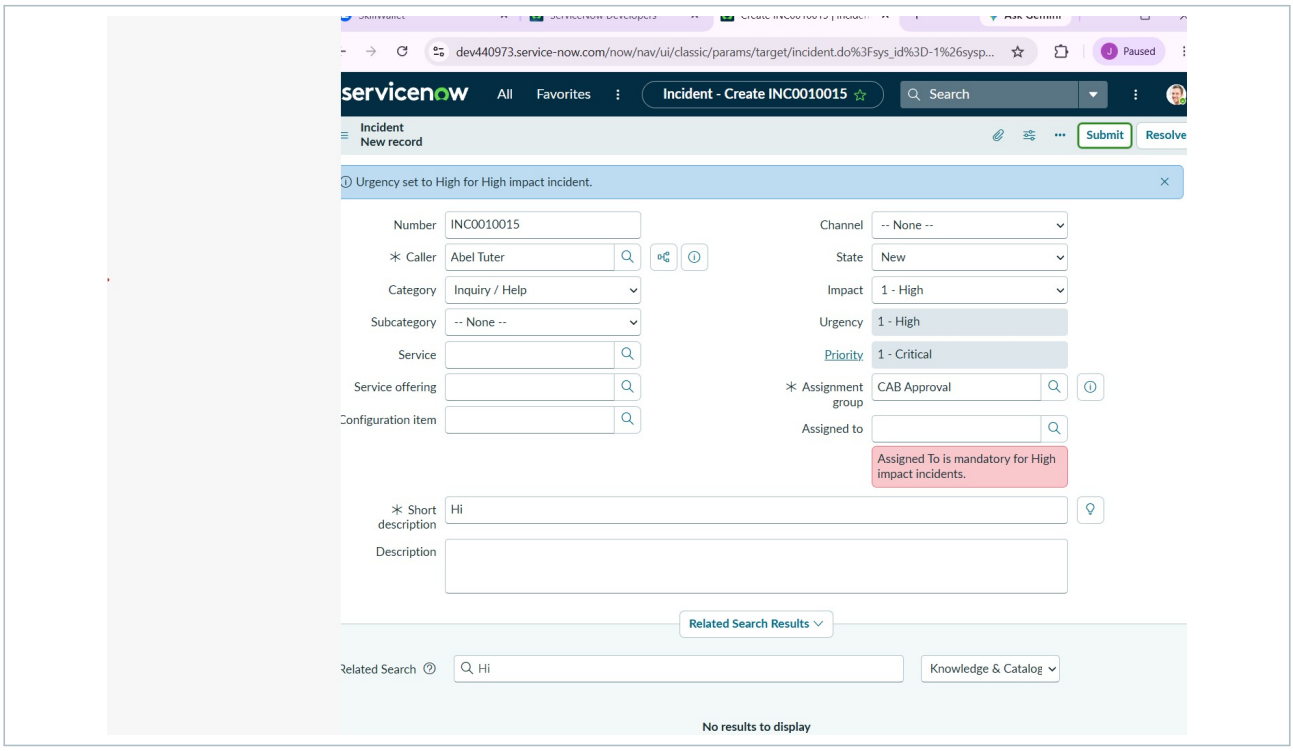



*Figure 1: New incident with Impact = High - Urgency auto-set to High, info message shown, and Assigned to flagged as mandatory for High impact incidents*

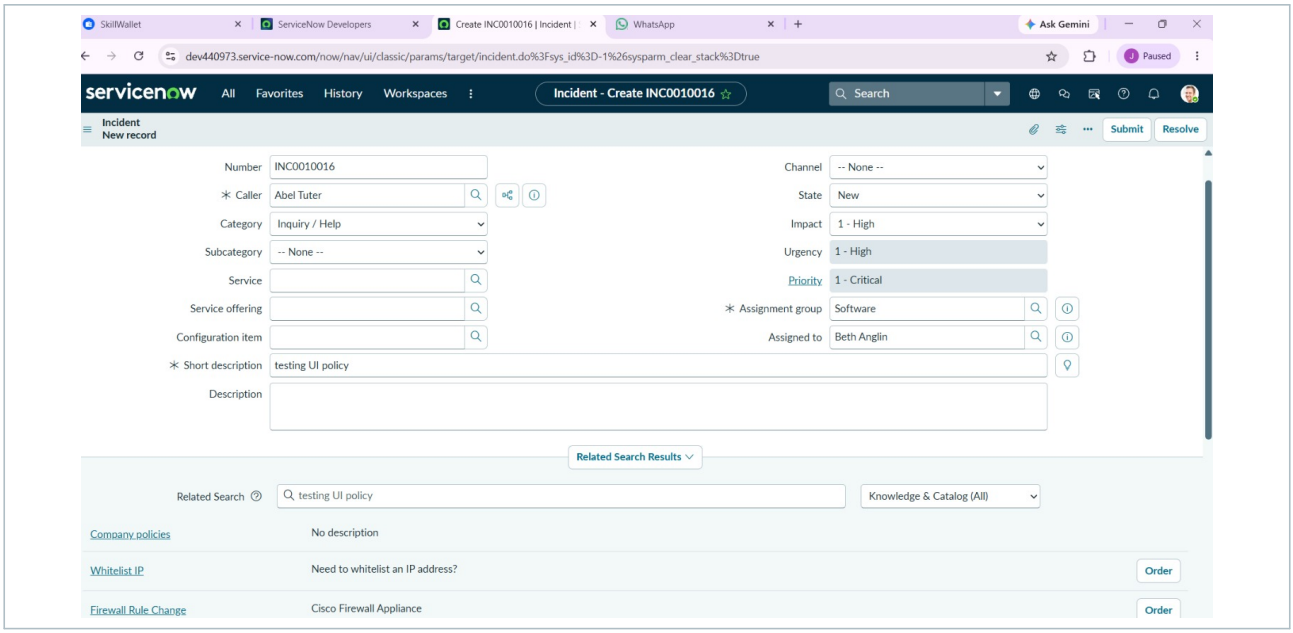



*Figure 2: After filling Assignment group and Assigned to, the new incident (Impact 1 - High, Priority 1 - Critical) is ready to submit*

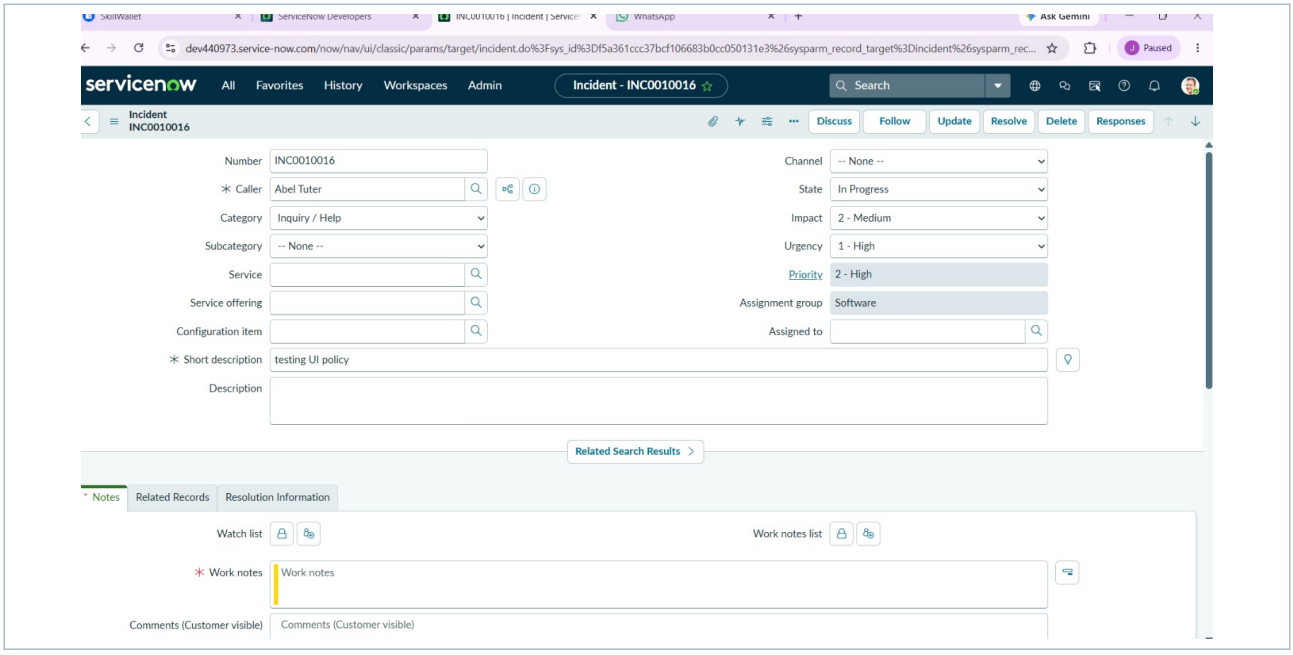



*Figure 3: Saved incident INC0010016 in progress with Priority and Assignment group fields locked (read-only) by UI Policy*

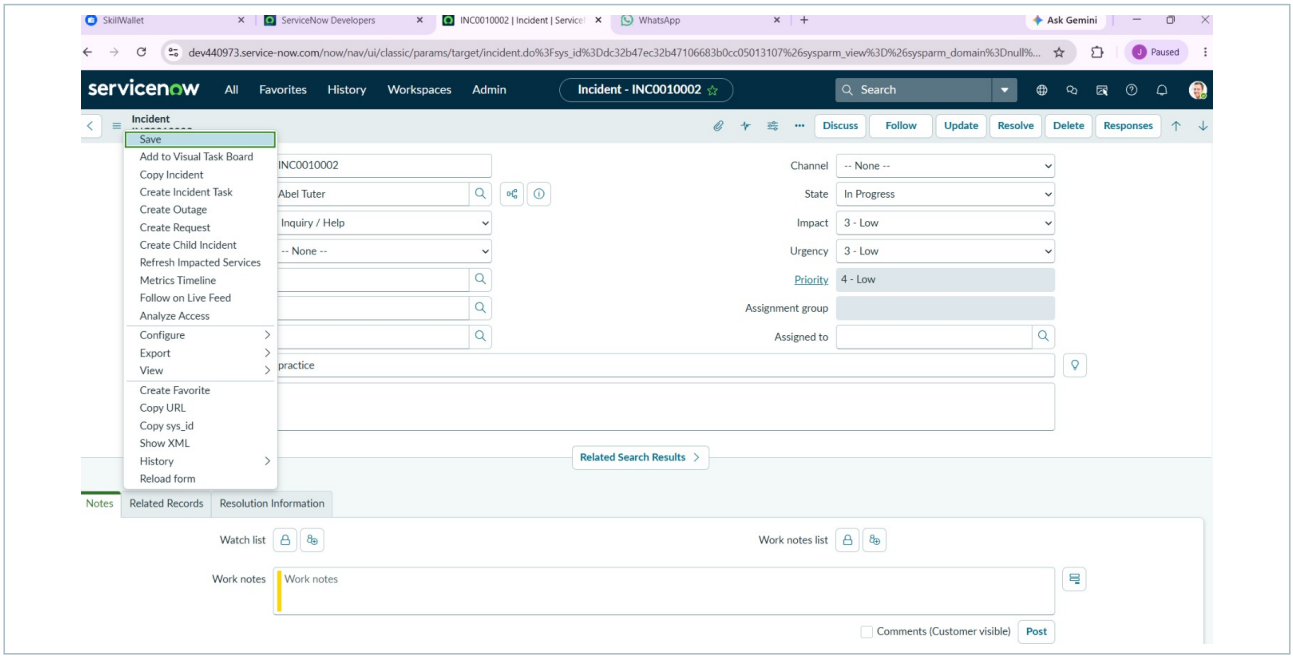



*Figure 4: Low impact / Low urgency incident (INC0010002) with Priority 4 - Low and Assignment group read-only; form menu showing Save*

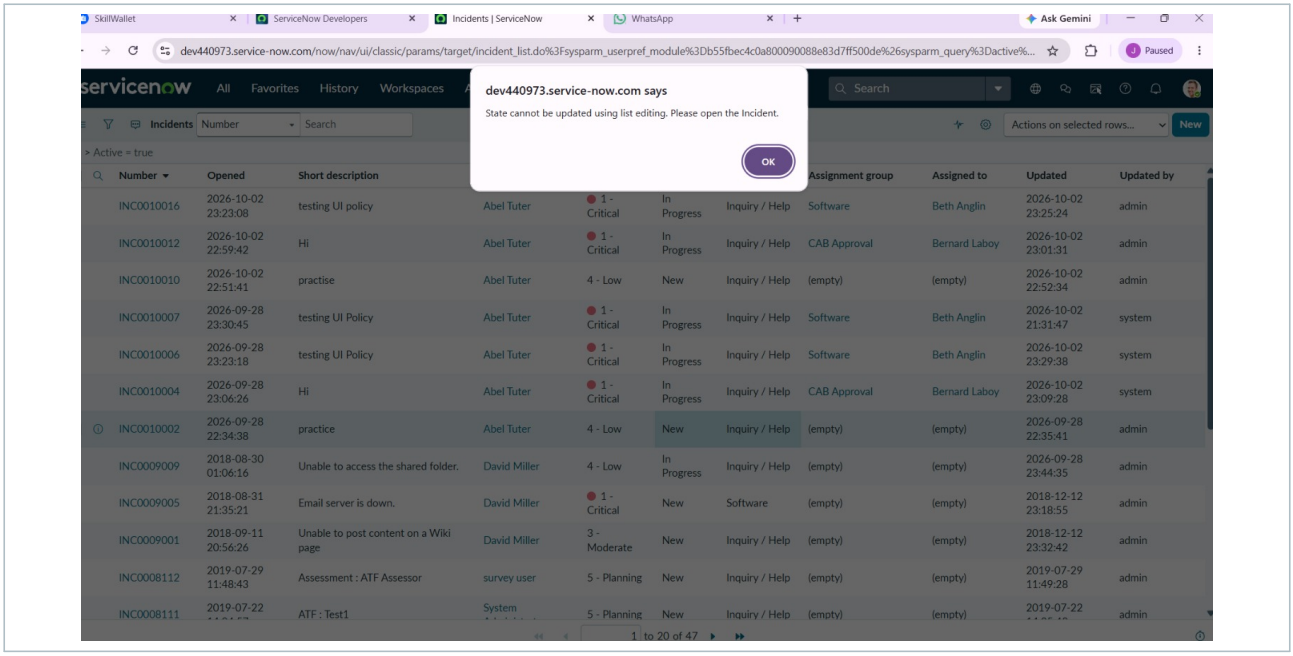



*Figure 5: Client Script alert in the incident list - "State cannot be updated using list editing. Please open the Incident."*

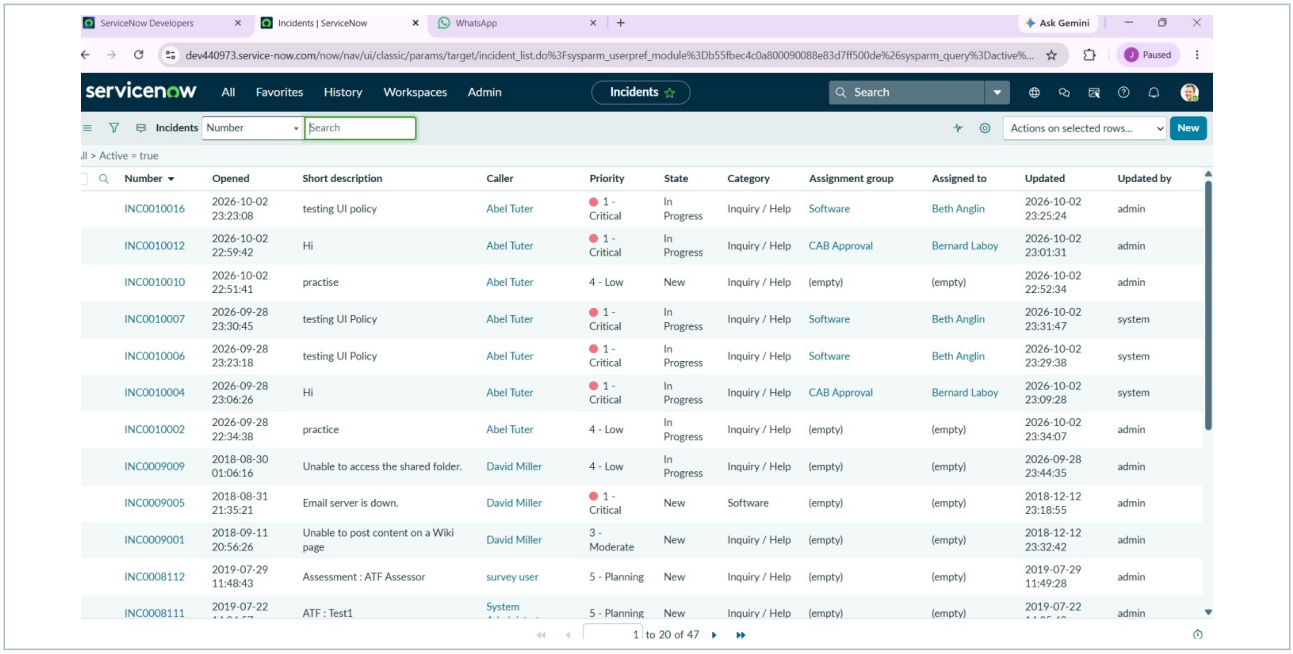


*Figure 6: Incident list view showing the resulting records with their priority, state and assignment details*

**Outcome:** the Incident form now enforces required data, adapts dynamically to field values and stops invalid actions, producing cleaner records.

# 8. Advantages & Disadvantages

| Advantages | Disadvantages |
| --- | --- |
| Improves data quality and consistency | Client-side rules can be bypassed via imports, APIs or integrations |
| Instant feedback without a server round-trip | Too many scripts may slow form loading |

| Advantages | Disadvantages |
|---|---|
| Low-code UI Policies are easy to maintain | Poorly written scripts can conflict with each other |
| Reduces rework and speeds up resolution | Needs testing across UI types (classic, workspace, mobile) |

## 9. Conclusion

The project successfully implemented UI Policies and Client Scripts on the Incident table of ServiceNow. Together they make forms smarter: fields are mandatory, hidden or read-only at the right time, values are auto-populated, and invalid submissions are blocked. The result is better data integrity, fewer manual corrections and a smoother experience for both callers and support agents.

## 10. Future Scope

- Add server-side Business Rules and Data Policies to protect data coming from imports and APIs.
- Extend the approach to Problem, Change and Service Request tables.
- Use Catalog Client Scripts and UI Actions for service catalog items.
- Integrate AI-driven suggestions for category and priority.
- Add automated testing with the Automated Test Framework (ATF).

## 11. Appendix

### Source Code (sample)

```
// onChange Client Script - Table: Incident, Field: Category
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') return;
    if (newValue == 'hardware') {
        g_form.setValue('impact', '2');
        g_form.showFieldMsg('category', 'Hardware incident: impact set to Medium.', 'info');
    }
}

// onSubmit Client Script - Table: Incident
function onSubmit() {
    var desc = g_form.getValue('short_description');
    if (desc.length < 10) {
        g_form.addErrorMessage('Short description must be at least 10 characters.');
        return false;   // stop submission
    }
}
```

### Dataset Link

Not applicable - this project is configured directly on the ServiceNow platform and uses no external dataset.